Lab 4 - Pętle

Pętla: **for**

- Służy do iteracji po sekwencjach lub zakresach.
- Wykonywana jest liczba iteracji (powtórzeń) wynikająca z długości sekwencji
- W Pythonie często używa się jej z funkcją **range()**.

Składnia:

```
for i in range(start, stop, step):
    # instrukcje

```

```
# Od 0 do 4
for i in range(5):
    print(i)  # 0 1 2 3 4

# Od 2 do 10 co 2
for i in range(2, 11, 2):
    print(i)  # 2 4 6 8 10
```

Sekwencje

- Sekwencje liczbowe możemy generować w Pythonie przy użyciu range
  - **range()** to wbudowana funkcja, która generuje sekwencję liczb całkowitych.
  - Najczęściej używa się jej w pętlach for, aby określić, ile razy pętla ma się wykonać.
  - Zwraca **obiekt** typu **range**, który jest **iterowalny** (można po nim przechodzić w pętli).
  - Nie tworzy od razu listy wszystkich liczb.
  - Podając ujemny krok (step) otrzymujemy sekwencję malejącą.

Składnia:

```
range(start, stop, step):
```

- Jeśli chcemy sprawdzić czy pewna wartość znajduje się w zbiorze (np. sekwencji) możemy to zrobić korzystając z operatora **in**. Wynikiem operacji jest wartość logiczna (jak przy operatorach porównania):

```
x = 2
print (x in range(10))
```

wynik: `True`

- Operator in można np. stosować do sprawdzenia występowania znaku w tekście:

```
x = "Hello world!"
print ("e" in x)
```

Pętla: **while**

- Wykonuje instrukcje dopóki warunek logiczny w nagłówku pętli jest prawdziwy.
- Nie jest z góry określona liczba powtórzeń (może zależeć od operacji w pętli)

Składnia:

```
while warunek:
    # instrukcje
```

Instrukcje sterujące w pętli

- Instrukcja **continue** - powoduje pominięcie bieżącej iteracji (pomijamy wszystkie instrukcje następujące po continue) i przejście do następnej.
- Instrukcja **break** – powoduje przerwanie działania pętli niezależnie od innych warunków (długości sekwencji lub warunku pętli while)
  - Instrukcję break czasem stosuje się w połączeniu z nieskończoną pętlą while (while True:) gdy nie wiemy ile powtórzeń wykonamy bo zależą np. od wczytywanych danych.

```
for i in range(n):
    #instrukcje wykonane
    continue
    #instrukcje pomijane
```

```
while n < 100:
    #instrukcje wykonywane
    if n==50:
        break #przy n==50 przerywamy pętlę
```

Zagnieżdżone pętle

- Pętle (niezależnie od rodzaju) możemy zagnieżdżać w innych pętlach – wtedy cały cykl pętli zagnieżdżonej jest wykonywany w każdej iteracji pętli zewnętrznej

```
n=10

for i in range(n):
    while i>0:
        print(i, end=" ")
        i-=1
    print()
```

```
1
2 1
3 2 1
4 3 2 1
5 4 3 2 1
6 5 4 3 2 1
7 6 5 4 3 2 1
8 7 6 5 4 3 2 1
9 8 7 6 5 4 3 2 1
```

- Uwaga: dla polecenia print możemy przekazać atrybut **end**, który zmienia domyślny sposób zakończenia linii (przejście do nowej linii). W przypadku podania spacji (" ") spowoduje to wyświetlenie wartości w jednej linii. Pominięcie atrybutu end skutkuje domyślnym przejściem do nowej linii.

Formatowanie wyświetlania liczb

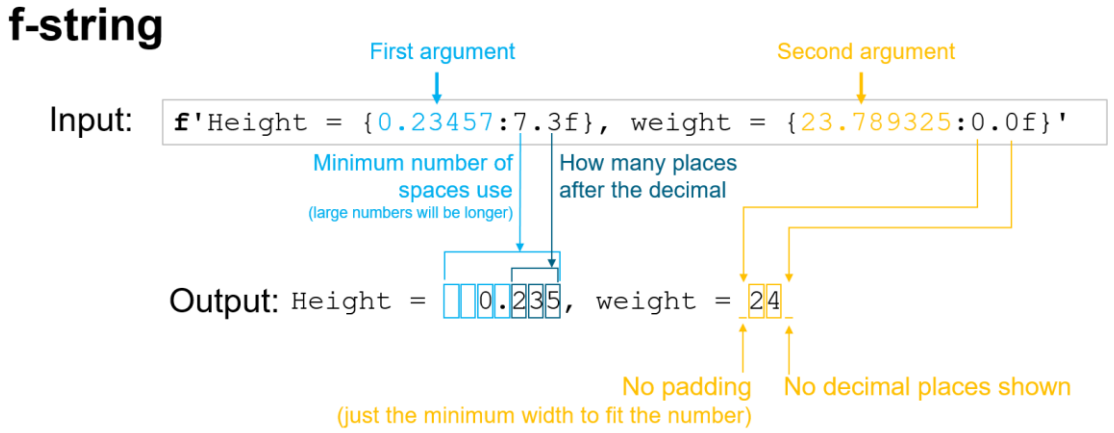
- Poznaliśmy już sposób wypisywania wartości zmiennych w połączeniu z tekstem, korzystając z tzw. f-string:

```
print(f'Wartość x wynosi {x}')
```

- Użycie f-string umożliwia nam zaawansowane formatowanie sposobu wyświetlania wartości poprzez określenie typu prezentacji danych po znaku **:**

```
x = 2
y = 1/3
print(f'Wartość x wynosi {x:03d}, a wartość y wynosi {y:10.3f}')
```

- Składnia formatowania f-string:



- Pełna lista sposobów formatowania dostępna np. na: https://www.w3schools.com/python/python_string_formatting.asp

Źródło: https://www.practicaldatascience.org/notebooks/other/fstrings.html [online: 23.10.2025]